LAPORAN PRAKTIKUM WEB SERVICE

SESI 09: MEMECAH APLIKASI MENJADI MICROSERVICES



DISUSUN OLEH

Bambang Herlambang

NRP 5803024019

Praktikum Individu

28 Oktober 2026

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS KATOLIK WIDYA MANDALA

SURABAYA

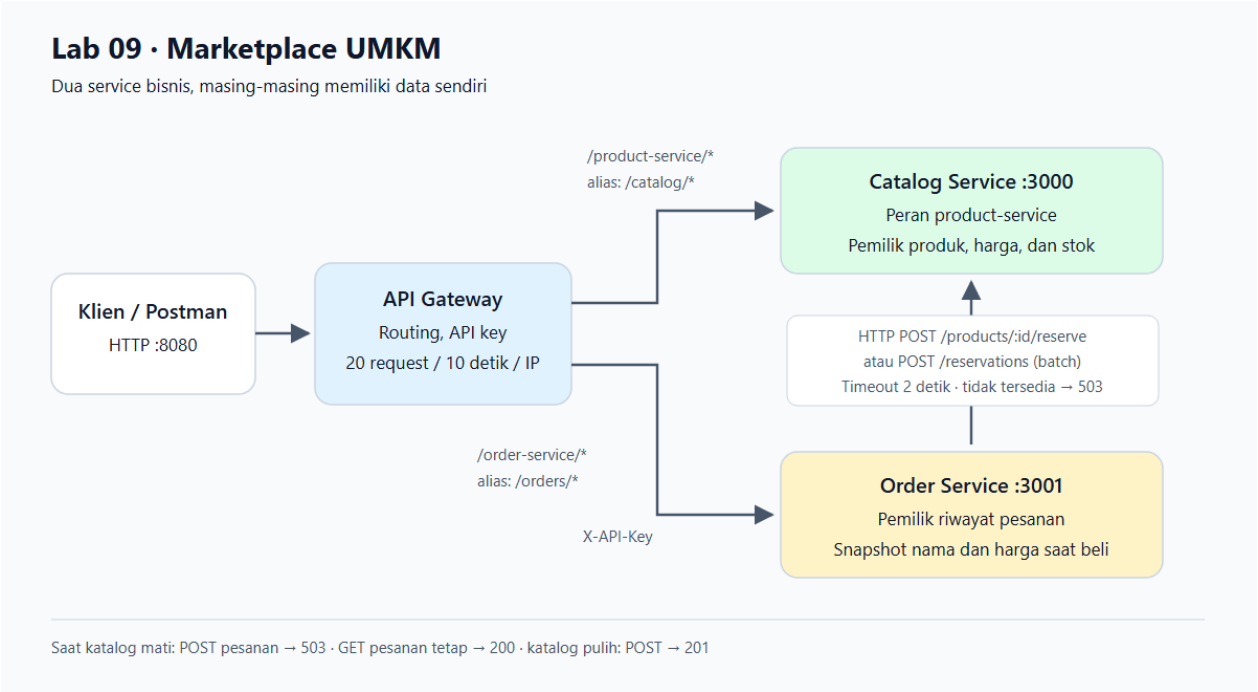
2026

1. Tujuan dan ruang lingkup

Praktikum ini memisahkan pengelolaan produk dan stok dari pengelolaan pesanan. Catalog Service menjadi satu-satunya pemilik data produk, sedangkan Order Service menyimpan riwayat pesanan dan meminta reservasi stok melalui HTTP. API Gateway menjadi pintu masuk bersama untuk kedua service.

Pengerjaan dilakukan secara individu pada repository khusus Lab 09, prakwebservice-lab09-microservices. Gateway dipakai ulang dari implementasi Lab 08. Contoh produk memakai konteks Marketplace UMKM Kelompok D; laporan ini mencatat implementasi praktikum individu dan pemetaan fiturnya ke rancangan proyek.

Nama folder `catalog-service` dipertahankan dari kode sumber dan berperan sebagai *product-service* pada materi. Data memakai ID `prod-001` dan body pesanan `customerName/items`, menggantikan contoh ID angka dan `productId/qty` tanpa mengubah prinsip pemisahan service.



Gambar 1. Arsitektur microservices: data produk dan pesanan dimiliki service yang berbeda. Reservasi stok dilakukan melalui HTTP dengan timeout dua detik.

Kriteria utama. Pesanan sukses mengurangi stok; kegagalan katalog menghasilkan POST pesanan `503`; GET pesanan tetap `200`; dan pembuatan pesanan pulih menjadi `201` setelah katalog hidup kembali.

2. Monolit, batas service, dan kepemilikan data

2.1 Baseline sebelum dipisah

Folder `monolith` menyediakan contoh satu proses yang menyimpan array produk dan pesanan. Handler `POST /orders` membaca harga, memeriksa stok, dan langsung mengurangi `product.stock`. Ketika proses ini mati, akses produk dan pesanan berhenti bersama.

```
npm install
npm run setup
npm run start:monolith
```

Uji `GET /products`, buat pesanan dua kopi, lalu baca `GET /products/prod-001`: stok awal 40 menjadi 38. Hentikan monolit sebelum menjalankan katalog pada port 3000.

2.2 Pemetaan ke proyek Marketplace UMKM

Service	Data dan tanggung jawab	API dan komunikasi
Catalog / Product	Produk, kategori, penjual, harga, stok tersedia. Satu-satunya pemilik stok.	Daftar/detail produk, GraphQL, reservasi satu produk dan batch. Dipanggil Order Service saat membuat pesanan.
Order	Pelanggan, jumlah barang, total, status, waktu, dan salinan nama/harga saat pembelian.	Daftar/detail/buat pesanan. Memanggil reservasi katalog lewat HTTP; tidak mengimpor data katalog.
API Gateway	Konfigurasi route, API key, dan penghitung rate limit; tanpa data bisnis.	Meneruskan request klien ke service setelah pemeriksaan gateway.

Tabel 1. Batas service berdasarkan data dan tanggung jawab.

2.3 Alur sesudah dipisah

1. Klien mengirim pesanan melalui gateway dengan API key.
2. Order Service memvalidasi body dan menggabungkan quantity produk yang berulang.
3. Order Service meminta reservasi ke katalog melalui HTTP.
4. Katalog memvalidasi seluruh stok sebelum menguranginya, lalu mengembalikan salinan produk.
5. Order Service menghitung harga dari respons katalog dan menyimpan pesanan.

Salinan nama dan harga membuat riwayat pesanan tidak berubah ketika data produk berikutnya berubah. Data tetap in-memory dan direset ketika proses pemiliknya dimulai ulang.

3. Kontrak API dan implementasi

3.1 Route melalui gateway

Default gateway adalah `http://127.0.0.1:8080`; katalog pada port 3000 dan pesanan pada port 3001. Prefix gateway dihapus sebelum request diteruskan. Alias asal `/catalog/*` dan `/orders/*` tetap tersedia.

Method	Path pada gateway	Status normal
GET	`/product-service/products`	200
GET	`/product-service/products/:id`	200
POST	`/product-service/products/:id/reserve`	200
POST	`/product-service/reservations`	200
GET	`/order-service/orders`	200
GET	`/order-service/orders/:id`	200
POST	`/order-service/orders`	201

Tabel 2. Endpoint utama Lab 09. Reservasi dan route pesanan memerlukan `X-API-Key` pada gateway; pembacaan katalog tidak.

3.2 Reservasi dan penanganan error

Reservasi tunggal memakai `POST /products/:id/reserve` dengan body `{"qty":2}`. Pesanan dengan beberapa jenis produk memakai `POST /reservations`; seluruh item diperiksa sebelum mutasi sehingga penolakan satu produk tidak memotong stok produk lain.

Order Service membatasi panggilan katalog, termasuk pembacaan body, selama dua detik. Gangguan jaringan atau timeout menjadi `503`. Produk tidak dikenal diteruskan sebagai `404`; stok tidak cukup sebagai `409`; input tidak valid sebagai `400`. Respons upstream yang rusak ditolak agar tidak menjadi sumber harga pesanan.

3.3 Contoh body pesanan

```
{
  "customerName": "Bambang",
  "items": [
    { "productId": "prod-001", "quantity": 2 }
  ]
}
```

Harga kopi berasal dari katalog, yaitu Rp85.000 per unit. Dua unit menghasilkan total Rp170.000; kolom harga atau total yang dikirim klien tidak dipakai sebagai sumber perhitungan.

4. Menjalankan, Postman, dan prosedur pengujian

4.1 Menyiapkan aplikasi

Gunakan Node.js 22 atau lebih baru dari root repository:

```
npm install
npm run setup
npm start
```

Setup membuat key lokal dalam `gateway/.env`. Isi hanya variabel `apiKey` lokal di Postman; jangan menampilkan nilainya pada screenshot.

4.2 Collection Postman JSON

Import `docs/postman/lab09.postman_collection.json` dan environment `docs/postman/lab09.postman_environment.json`. Collection berisi 14 request dalam tiga kelompok: alur normal, input negatif, lalu kegagalan dan pemulihan. Semua request klien menuju gateway.

Alur normal membuktikan tiga produk tersedia, POST pesanan diterima, stok turun dua, dan riwayat tersimpan. Skenario negatif menguji quantity nol, produk tidak dikenal, stok tidak cukup, serta penolakan reservasi. Kelompok kegagalan dijalankan manual dengan intervensi untuk mematikan dan menghidupkan katalog di antara request.

4.3 Otomatis dan manual

```
npm test
npm run test:lab09
```

Suite otomatis menggunakan assertion atas respons HTTP. Demo `test:lab09` menyiapkan tiga proses pada port lokal yang tersedia, merekam smoke test, menghentikan hanya katalog, lalu menghidupkannya kembali. Untuk stack yang sedang berjalan, `npm run test:smoke` membuat pesanan sungguhan dan mengurangi stok.

Uji kegagalan manual memakai tiga terminal:

```
node catalog-service/server.js
node order-service/server.js
node gateway/gateway.js
```

Hentikan hanya terminal katalog, lakukan POST dan GET pesanan, lalu hidupkan kembali katalog sebelum mengulangi POST. Launcher `npm start` menutup seluruh stack jika satu child keluar, sehingga launcher tersebut tidak dipakai untuk mematikan satu service secara manual. Rate limit gateway tetap 20 request per 10 detik per IP.

5. Hasil pengujian lokal

5.1 Ringkasan assertion dan smoke test

Log `docs/evidence/lab09/tests.txt` mencatat **67 test lulus dari 67, 0 gagal**. Skenario mencakup stok, produk berulang, batch, pesanan bersamaan, GraphQL, API key, route gateway, timeout, dan isolasi kegagalan.

Demo direkam pada **28 Oktober 2026 pukul 16:32:19 UTC+7**, dengan timestamp sumber `2026-10-28T09:32:19.961Z`. Tiga proses lokal terpisah menjalankan katalog, pesanan, dan gateway.

Pemeriksaan	Hasil aktual
Jumlah produk	3
POST pesanan	HTTP 201
Stok kopi sebelum / sesudah	40 / 38
Total pesanan	Rp170.000
Katalog berhenti: POST / GET pesanan	HTTP 503 / HTTP 200
Katalog hidup kembali: POST pesanan	HTTP 201
Jumlah pesanan setelah pemulihan	2, termasuk pesanan sebelum gangguan

Tabel 3. Ringkasan bukti smoke test dan kegagalan service.

5.2 Asal screenshot

Empat screenshot pada halaman berikut merupakan tangkapan browser atas halaman bukti otomatis `docs/evidence/lab09/index.html`. Halaman itu menampilkan request dan respons HTTP aktual dari skrip Node.js melalui gateway lokal. **Gambar tersebut bukan screenshot Postman.** Collection Postman disediakan untuk mengulangi pengujian secara manual.

Header request yang berisi API key tidak disimpan pada JSON, log, atau halaman bukti. Port dipilih otomatis, sehingga berbeda dari port default aplikasi. Screenshot dan JSON harus berasal dari demo yang sama; menjalankan ulang skrip menuntut pengambilan screenshot baru.

5.3 Bukti smoke test

Tiga produk berhasil dibaca melalui gateway. Pesanan dua kopi diterima dengan `201`; stok berkurang dari 40 menjadi 38, dengan total Rp170.000.



LULUS

01 / Smoke test melalui gateway

Bambang Herlambang · 5803024019 · Lab 09

Tiga produk; pesanan HTTP 201; stok 40 menjadi 38; total Rp170000.

Eksekusi: 2026-10-28T09:32:19.961Z

`GET http://127.0.0.1:55880/product-service/products`

HTTP 200 · 17 ms


```
{
  "service": "catalog-service",
  "data": [
    {
      "id": "prod-001",
      "name": "Kopi Arabika Gayo",
      "category": "kuliner",
      "price": 85000,
      "stock": 40,
      "seller": "Koperasi Kopi Gayo"
    },
    {
      "id": "prod-002",
      "name": "Tas Anyaman Pandan",
      "category": "kerajinan",
      "price": 120000,
      "stock": 15,
      "seller": "Rumah Anyaman Nusantara"
    },
    {
      "id": "prod-003",
      "name": "Kaos Batik UMKM",
      "category": "fashion",
      "price": 95000,
      "stock": 30,
      "seller": "Batik Lokal Mandiri"
    }
  ]
}
```


`POST http://127.0.0.1:55880/order-service/orders`

Request body


```
{
  "customerName": "Bambang Lab 09",
  "items": [
    {
      "productId": "prod-001",
      "quantity": 2
    }
  ]
}
```


HTTP 201 · 84 ms


```
{
  "data": {
    "id": "ord-ca5ac52c-cb12-4198-8385-d8d60cdea0f6",
    "customerName": "Bambang Lab 09",
    "items": [
      {
        "productId": "prod-001",
        "name": "Kopi Arabika Gayo",
        "quantity": 2,
        "unitPrice": 85000,
        "subtotal": 170000
      }
    ],
    "total": 170000,
    "status": "pending",
    "createdAt": "2026-10-28T09:32:20.061Z"
  }
}
```


`GET http://127.0.0.1:55880/product-service/products/prod-001`

HTTP 200 · 6 ms


```
{
  "data": {
    "id": "prod-001",
    "name": "Kopi Arabika Gayo",
    "category": "kuliner",
    "price": 85000,
    "stock": 38,
    "seller": "Koperasi Kopi Gayo"
  }
}
```


Sumber: npm run test:lab09. Ini halaman bukti pengujian otomatis; bukan tangkapan layar Postman. Header API key tidak disimpan.

Gambar 2. Respons HTTP aktual smoke test melalui gateway lokal.

5.4 POST saat katalog berhenti

Skrip menghentikan hanya proses katalog. Gateway dan Order Service tetap berjalan. Pembuatan pesanan gagal dengan `503`, disertai pesan yang jelas.

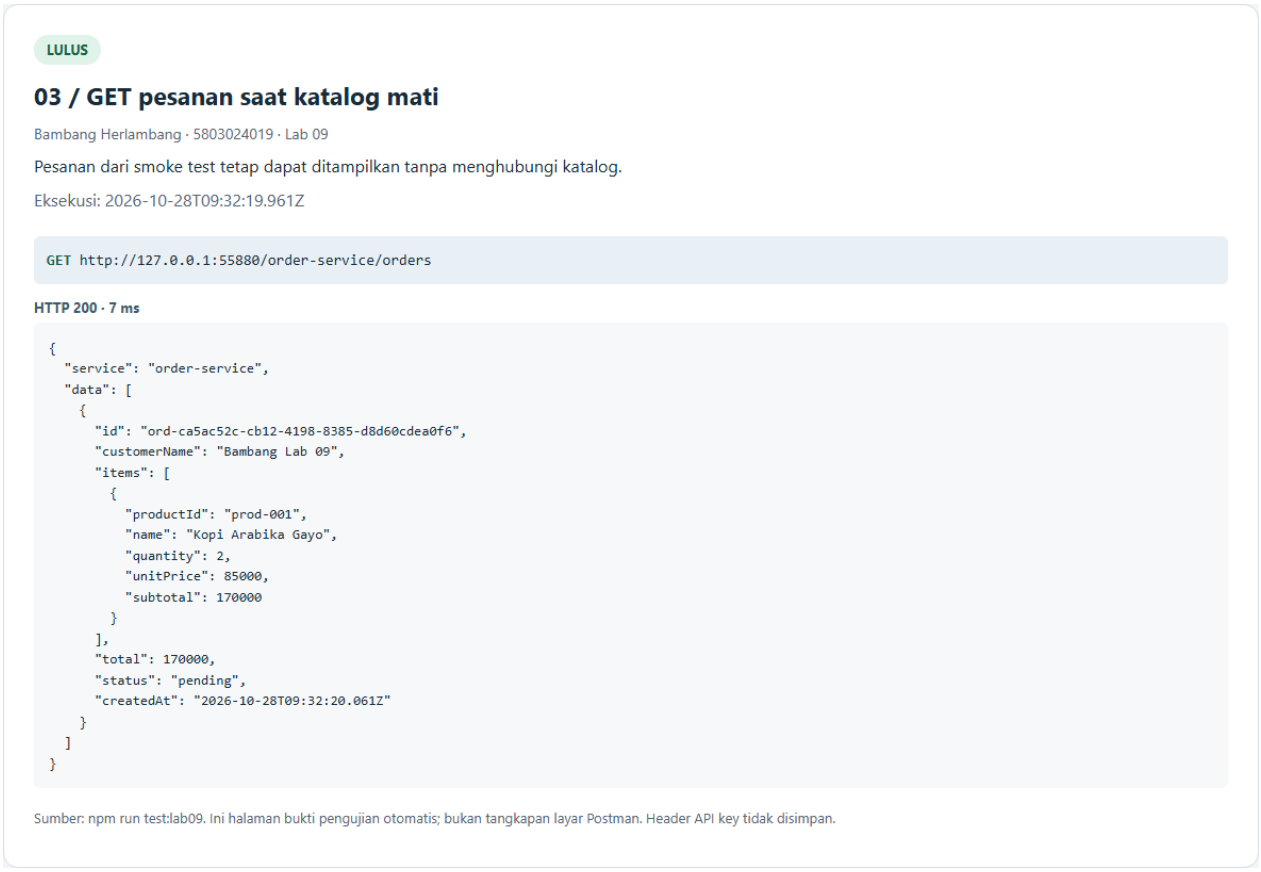


Gambar 3. POST pesanan menjadi 503 saat Catalog Service tidak tersedia.

POST yang ditolak tidak menambahkan pesanan. Pemeriksaan berikutnya membaca daftar pesanan langsung dari Order Service untuk membuktikan bahwa kegagalan katalog tidak menghentikan seluruh sistem.

5.5 GET saat katalog masih berhenti

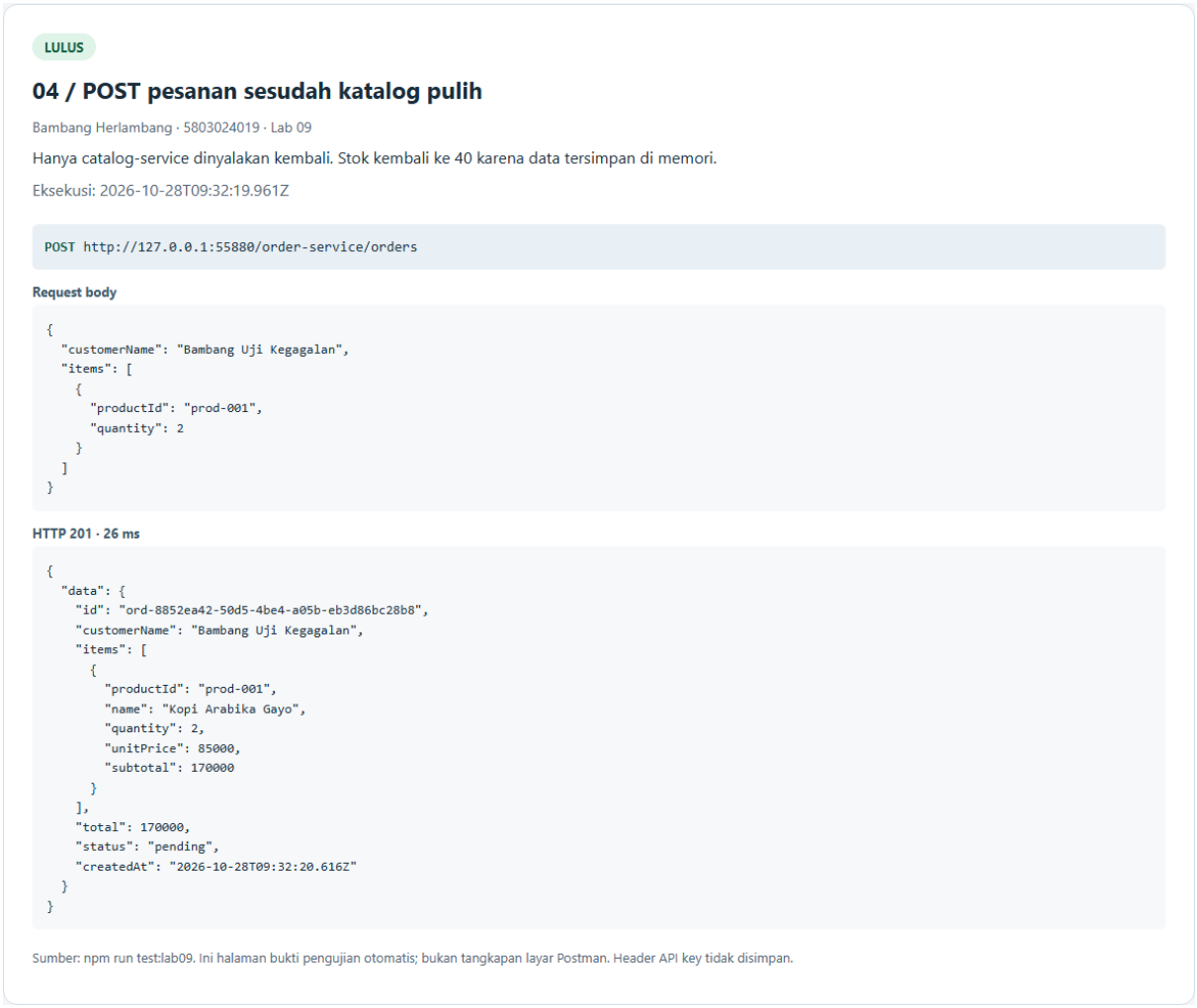
Order Service tetap dapat menampilkan pesanan dari smoke test tanpa menghubungi katalog. Respons 200 berikut membuktikan riwayat pesanan tetap tersedia pada proses yang sama.



Gambar 4. GET pesanan tetap 200 dan memuat pesanan sebelum gangguan.

5.6 POST setelah katalog hidup kembali

Hanya katalog yang dihidupkan kembali. Fixture katalog dimulai dari stok 40; POST pesanan baru diterima dengan 201 dan stok menjadi 38. Order Service tetap menyimpan pesanan sebelum gangguan.

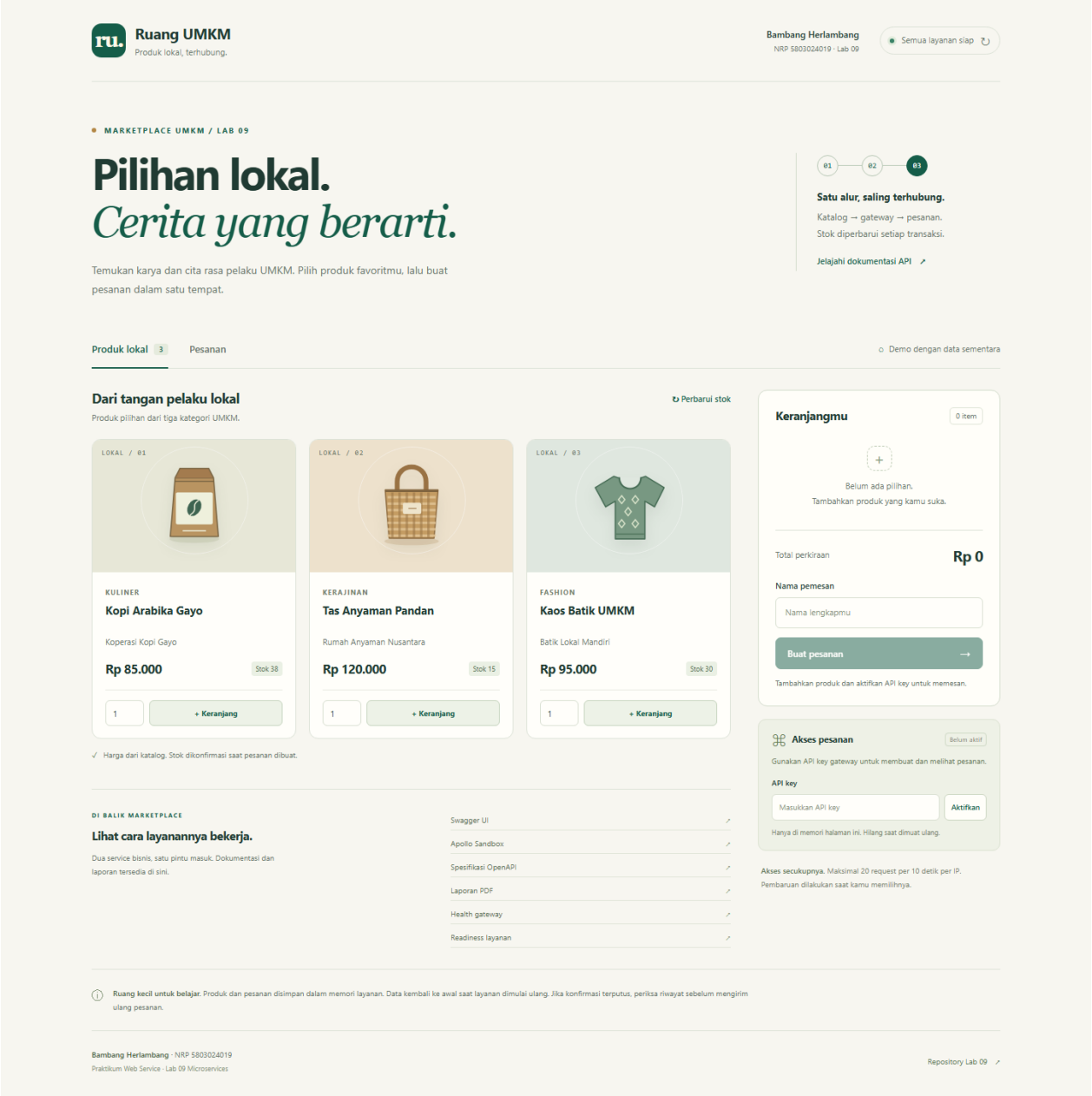


Gambar 5. Pembuatan pesanan pulih menjadi 201 setelah katalog berjalan kembali.

6. Frontend, dokumentasi interaktif, dan deployment

6.1 Frontend Ruang UMKM

Frontend pada route / menghubungkan katalog, keranjang, pembuatan pesanan, dan riwayat melalui gateway yang sama. Halaman menyediakan tiga produk, perubahan quantity, total perkiraan, nama pemesan, serta input API key untuk operasi pesanan. Harga akhir dan ketersediaan stok tetap ditentukan service, bukan nilai yang ditampilkan klien.

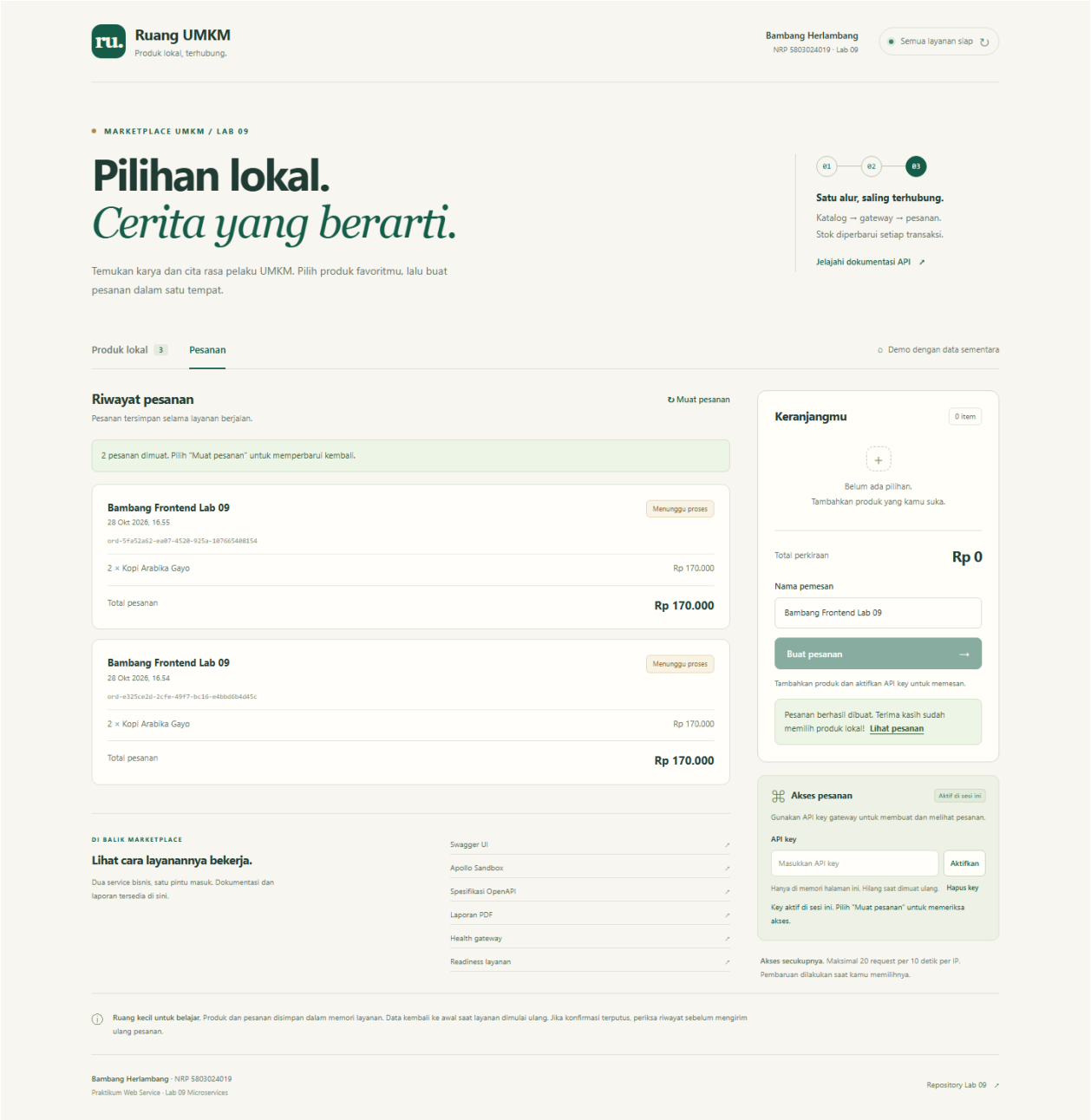


Gambar 6. Frontend lokal Ruang UMKM: daftar produk, keranjang, dan akses pesanan. Key belum diaktifkan pada tangkapan ini.

Tautan Swagger UI, Apollo Sandbox, OpenAPI JSON, laporan PDF, health, dan readiness tersedia dari halaman. API key hanya disimpan dalam memori sesi halaman; tidak ditulis ke localStorage atau sessionStorage.

6.2 Checkout dan riwayat pesanan

Pengujian browser lokal mencatat checkout dua kopi berhasil: stok yang terbaca sebelum transaksi adalah 38 dan setelah transaksi 36. Pengujian ini terpisah dari smoke test 40 menjadi 38 pada bagian sebelumnya. Halaman riwayat menampilkan pesanan, rincian barang, harga, dan status.

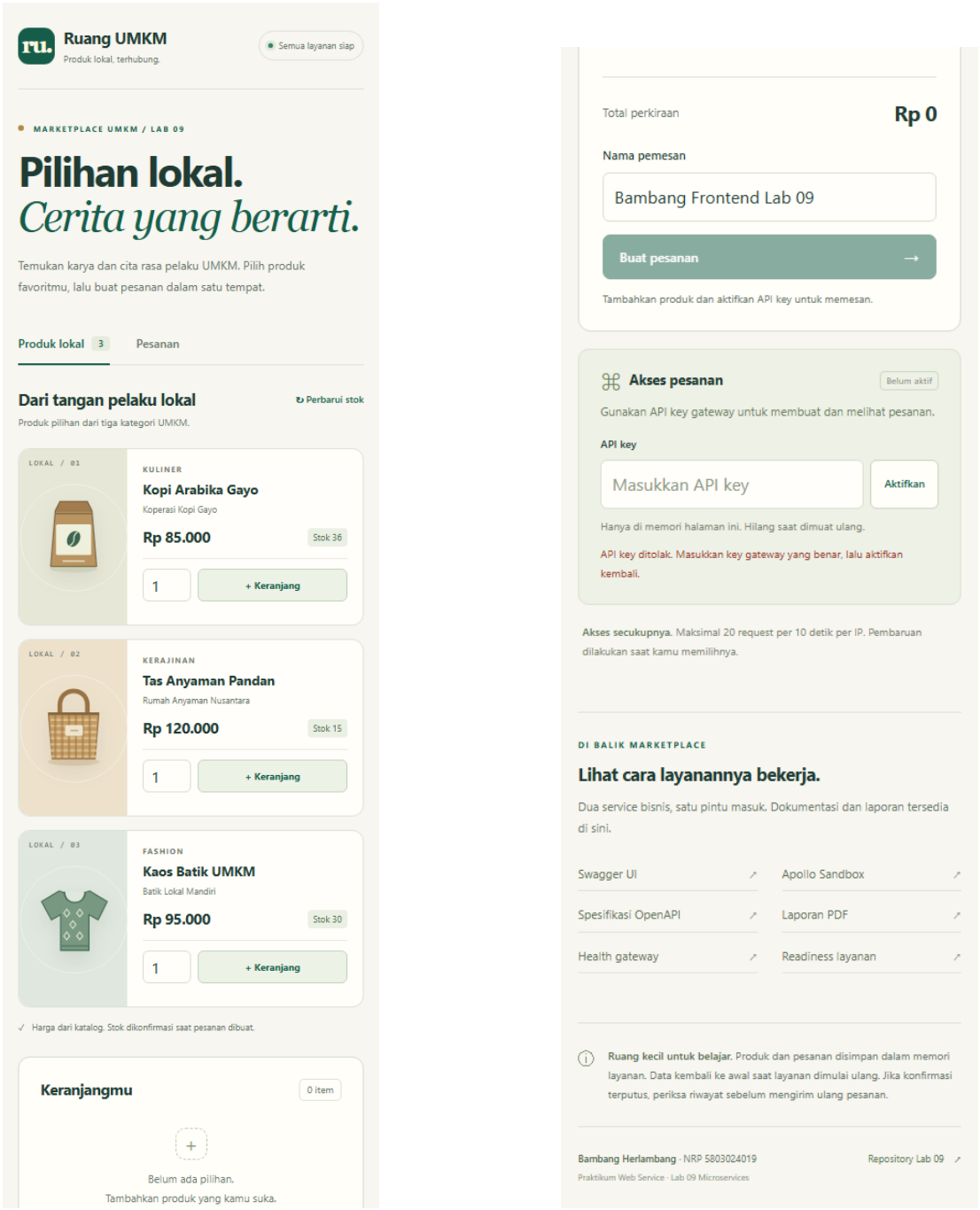


Gambar 7. Riwayat pesanan lokal setelah checkout melalui frontend. Nilai key tidak ditampilkan; halaman menggunakan kredensial sesi sementara.

Rekaman `docs/evidence/ui/ui-verification.json` memuat hasil checkout, penolakan API key salah, pengujian mobile, Swagger, dan pemuatan iframe Apollo. Pengujian browser tersebut tidak mencatat page error.

6.3 Tampilan mobile dan key tidak valid

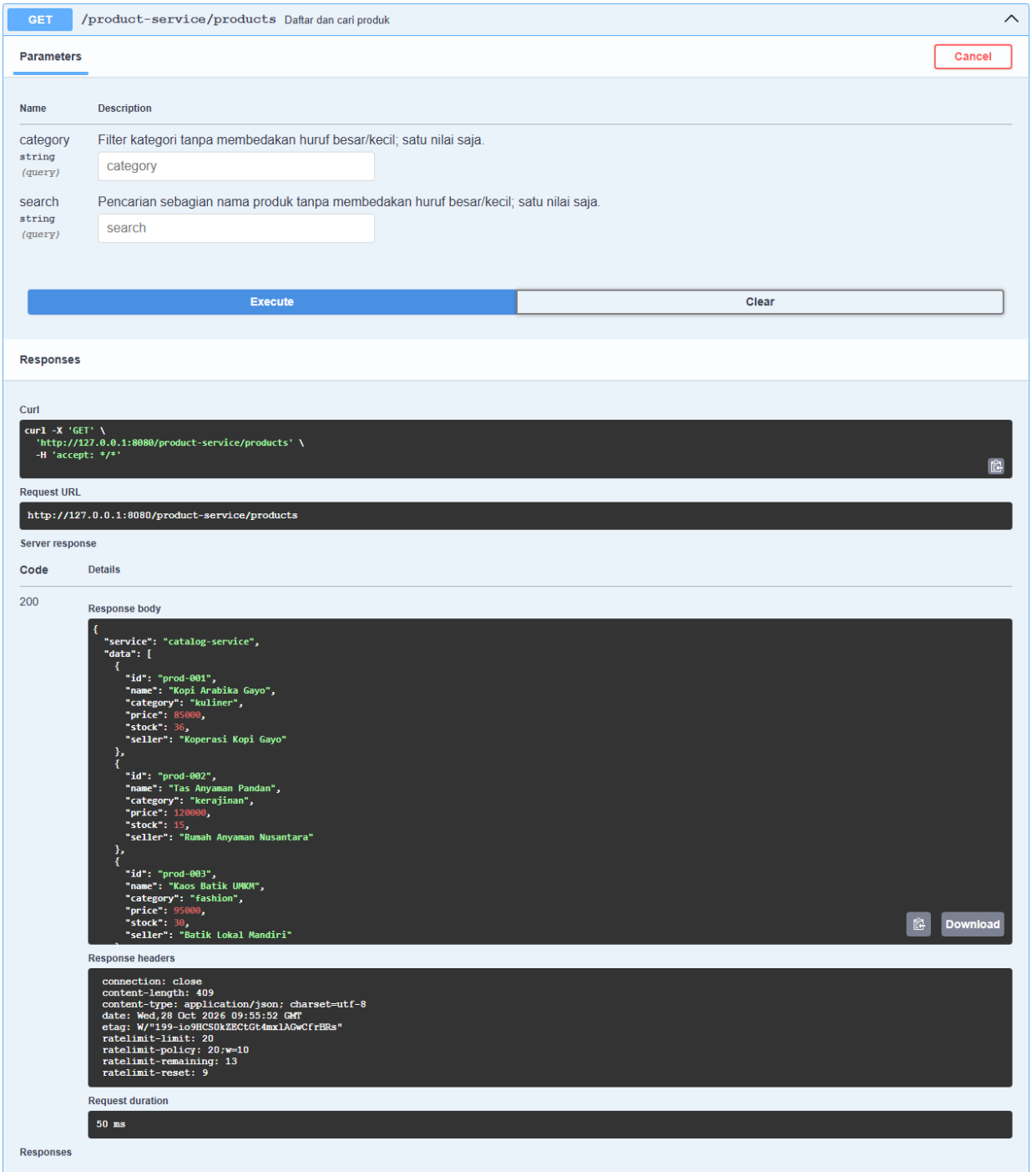
Pada viewport 390 piksel, verifikasi browser tidak menemukan overflow horizontal. Produk, keranjang, form pesanan, dan navigasi disusun ulang untuk layar sempit. Key salah ditolak dan halaman menampilkan pesan kegagalan tanpa menyimpan nilai key dalam browser storage.



Gambar 8. Dua potongan berurutan dari satu screenshot mobile lokal: katalog pada bagian atas dan form beserta pesan key ditolak pada bagian bawah. Isi gambar tidak diubah.

6.4 Swagger UI dan spesifikasi OpenAPI

Swagger UI tersedia pada `/docs/` dan memuat spesifikasi `/openapi.json`. Dokumentasi meliputi endpoint katalog, pesanan, reservasi, GraphQL, health, serta respons error. Fungsi *Authorize* memungkinkan key dikirim untuk operasi yang memerlukannya tanpa menanamkan nilai key dalam spesifikasi.

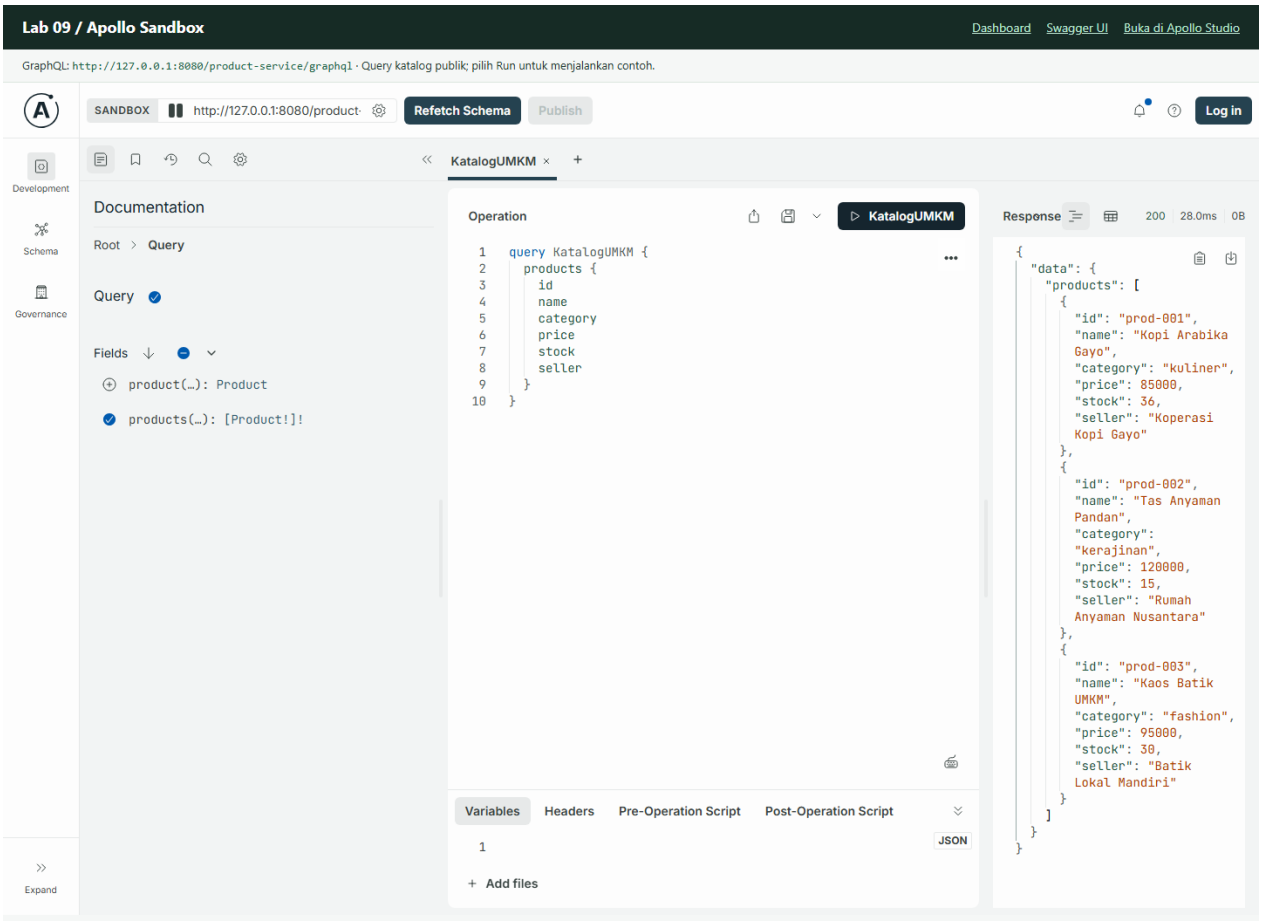


Gambar 9. Swagger Try it out pada gateway lokal mengembalikan HTTP 200 dan daftar produk. Area respons aktual ditampilkan; contoh respons statis di bawahnya dipotong agar tidak tertukar sebagai bukti eksekusi.

Halaman dokumentasi tidak memakai kuota request API. Request yang dibuat melalui Try it out tetap melewati route gateway, pemeriksaan key, dan rate limit yang sama dengan klien lain.

6.5 Apollo Sandbox dan GraphQL katalog

Apollo Sandbox tersedia pada `/sandbox`. Endpoint GraphQL gateway: `/product-service/graphql`. Halaman memakai Embedded Sandbox Apollo, menyediakan query awal katalog, dan dapat membuka endpoint yang sama di Apollo Studio. SDK tersebut memerlukan akses internet.



Gambar 10. Operasi KatalogUMKM dijalankan melalui tombol Run pada Apollo Sandbox lokal. Respons HTTP 200 menampilkan tiga produk beserta harga dan stok; panel respons terlihat di sisi kanan.

```
query KatalogUMKM {
  products { id name category price stock seller }
}
```

Hasil eksekusi tombol Run direkam dalam `docs/evidence/ui/apollo-query.json`: status 200 dan tiga produk. Query GraphQL katalog juga diverifikasi pada suite HTTP otomatis. Berbagi akses lintas origin dibatasi pada endpoint GraphQL katalog yang bersifat baca; kredensial pesanan tidak dibutuhkan untuk query tersebut.

6.6 Konfigurasi deployment Render Free

Deployment berjalan pada **service Render terpisah khusus Lab 09**, menggunakan repository dan branch `main` Lab 09. Blueprint `render.yaml` mendefinisikan satu instance Free yang menjalankan tiga proses terpisah: gateway, katalog, dan pesanan. Hanya gateway yang dipublikasikan; backend tetap pada loopback.

Konfigurasi	Nilai
Runtime / versi	Node.js 22
Nama service	`prakwebservice-lab09-microservices`
Region / plan	Singapore / Free
Build	`npm ci -omit=dev`
Start	`npm start`
Health check	`/readyz`
Gateway	Host `0.0.0.0`, port dari platform, `TRUST_PROXY=1`
Backend	Host loopback; katalog port 3000, pesanan port 3001
Key server	Digest SHA-256 melalui `API_KEY_SHA256`

Tabel 4. Konfigurasi deployment pada blueprint Lab 09.

Status live. Rekaman CLI Render pada `docs/evidence/deployment/deploy-status.json` menyatakan deployment awal `0e40b71` berstatus `live`, selesai pada 28 Oktober 2026 pukul **17:02:02 UTC+7**. Identitas service adalah `srv-dbgsfp6gekts738pdq4g`. Bukti ini adalah rekaman deployment awal, bukan klaim bahwa hash tersebut selalu menjadi commit terbaru.

Alamat gateway publik: `https://prakwebservice-lab09-microservices.onrender.com`.

Akses publik	Tautan melalui gateway HTTPS
Frontend	Dashboard Ruang UMKM
Swagger UI	/docs/
Apollo Sandbox	/sandbox
GraphQL	/product-service/graphql
Laporan PDF	/laporan.pdf
Sumber Overleaf	/laporan-latex.zip

Tabel 5. Tautan publik aplikasi dan berkas pengumpulan Lab 09.

6.7 Hasil verifikasi HTTPS publik

Perintah `npm run verify:deployment` selesai pada 28 Oktober 2026 pukul **17:02:52 UTC+7**. Sumbernya adalah `docs/evidence/deployment/verification.json` dan `verification.txt`. Pemeriksaan dilakukan terhadap service Lab 09, terpisah dari deployment Lab 08.

Skenario publik	Hasil aktual
Readiness gateway dan kedua service	`/readyz`: HTTP 200
Frontend, JavaScript, CSS	HTTP 200
Swagger UI dan Apollo Sandbox	HTTP 200
Unduh PDF dan ZIP LaTeX	HTTP 200; signature PDF/ZIP valid
Query GraphQL katalog	HTTP 200; 3 produk
Pesanan tanpa key	HTTP 401
POST pesanan dengan key sah	HTTP 201; total Rp170.000
Stok kopi pada smoke test publik	40 menjadi 38
GET riwayat pesanan dengan key	HTTP 200; pesanan sukses tersimpan

Tabel 6. Hasil verifier HTTP terhadap gateway Render yang aktif.

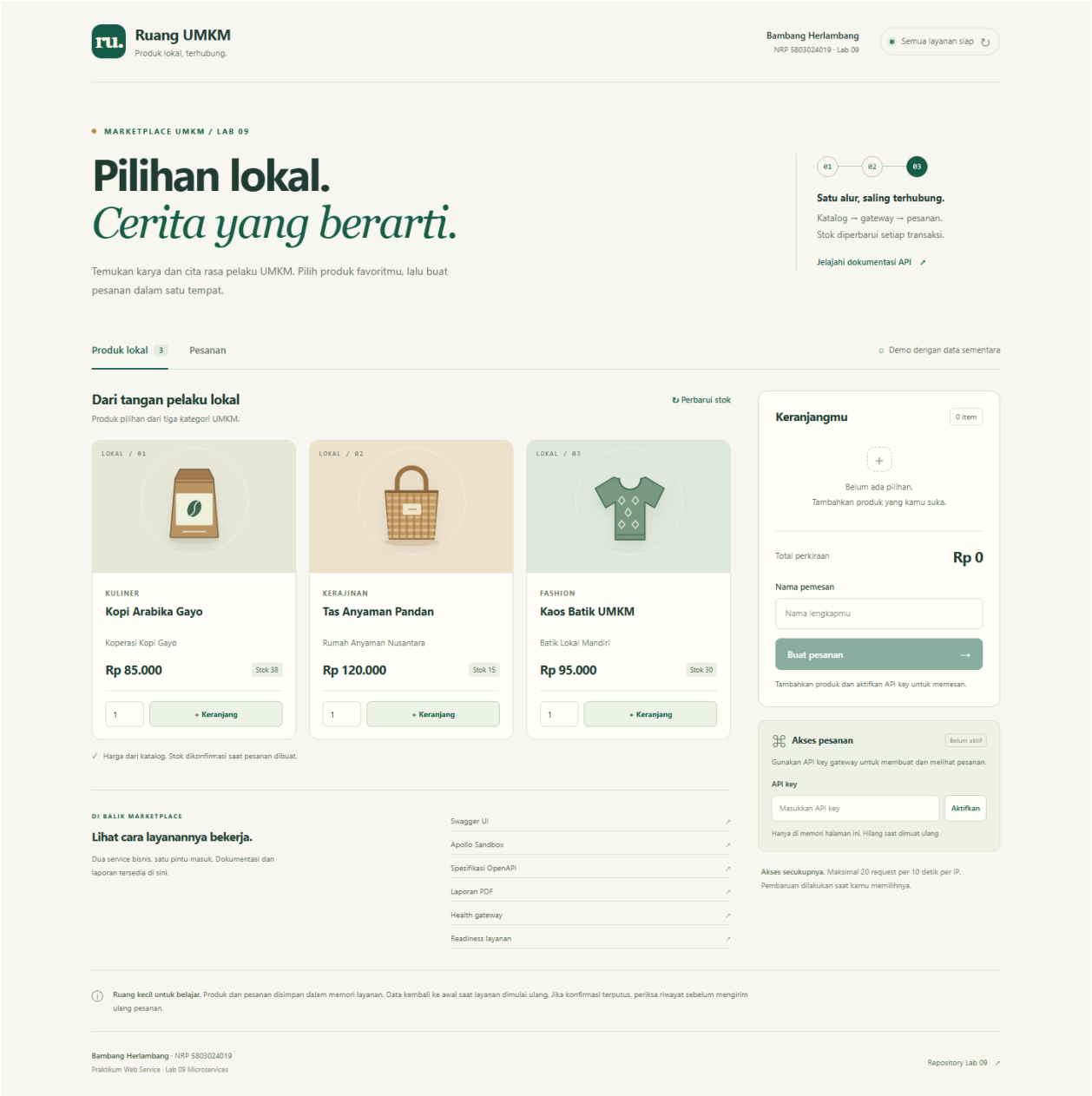
Verifikasi browser publik berikutnya dicatat pada pukul **17:03:29 UTC+7** dalam `docs/evidence/deployment/ui-verification.json`. Checkout frontend berhasil mengurangi stok dari 38 menjadi 36 dan menampilkan dua pesanan. Key salah ditolak; viewport 390 piksel tidak overflow horizontal; Swagger Try it out dan query Apollo menghasilkan HTTP 200. Rekaman browser tidak memuat page error dan key tidak disimpan dalam browser storage.

Empat screenshot publik pada halaman berikut berasal dari browser yang membuka URL Render tersebut. Screenshot Swagger menunjukkan respons server aktual, dan screenshot Apollo menunjukkan hasil operasi yang dijalankan melalui tombol Run. Sumber lengkap disimpan pada `docs/evidence/deployment/`; tidak ada screenshot dashboard Render yang direkayasa. Status live bersumber dari JSON CLI resmi Render.

Collection dan environment Postman tersedia dalam `docs/postman/`. Environment publik memakai base URL gateway deployment, sedangkan nilai key diisi lokal oleh klien. Data tetap in-memory; service yang dimulai ulang mengembalikan fixture produk dan menghapus pesanan di memorinya.

6.8 Frontend pada deployment publik

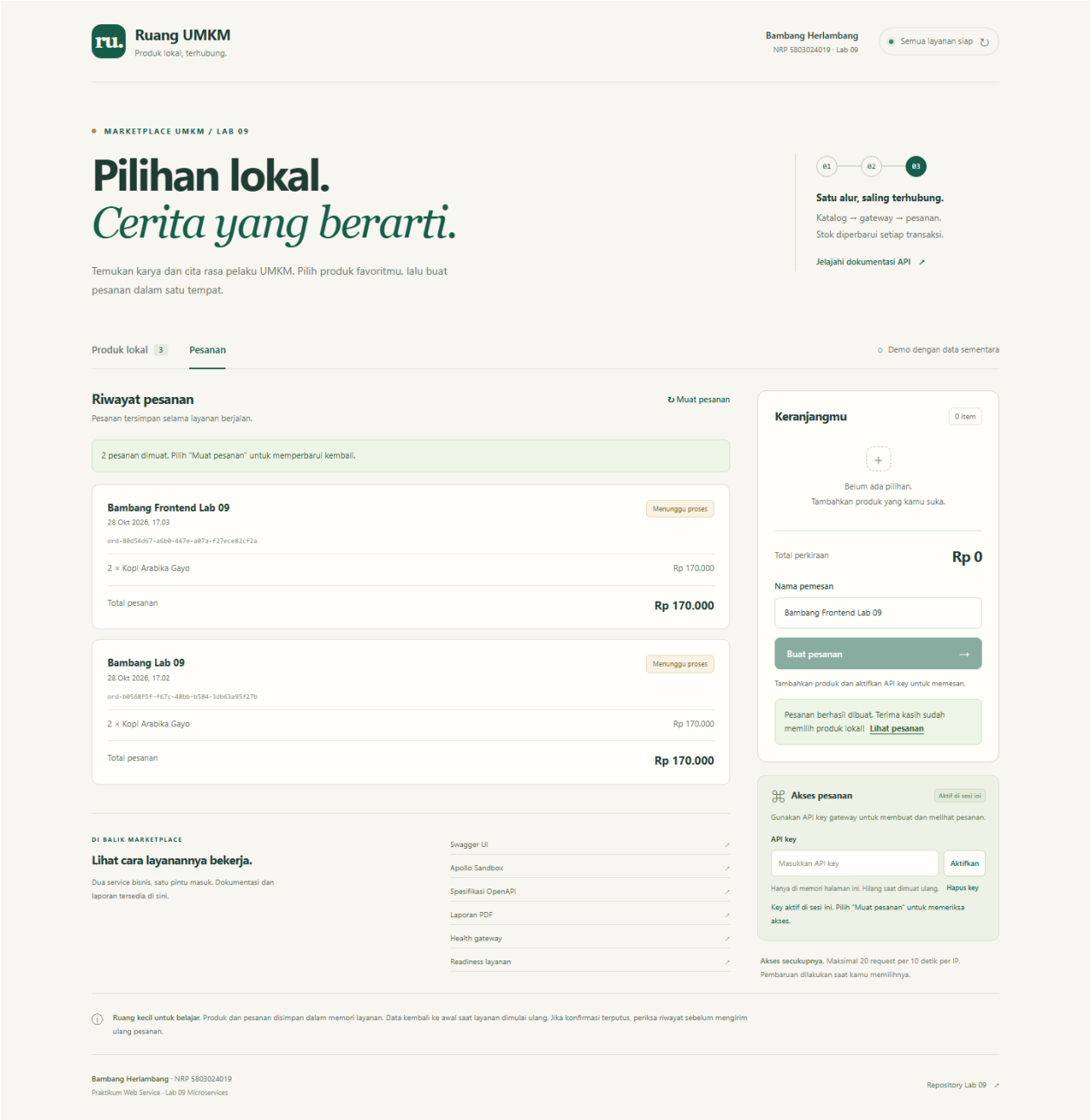
Halaman Ruang UMKM berikut dibuka melalui gateway HTTPS Render. Katalog menampilkan stok yang berasal dari API publik, dan navigasi mengarah ke dokumentasi serta laporan pada deployment yang sama.



Gambar 11. Frontend Ruang UMKM pada service Render Lab 09. Sumber tangkapan browser tercatat pada ui-verification.json dengan base URL HTTPS publik.

6.9 Checkout frontend pada deployment publik

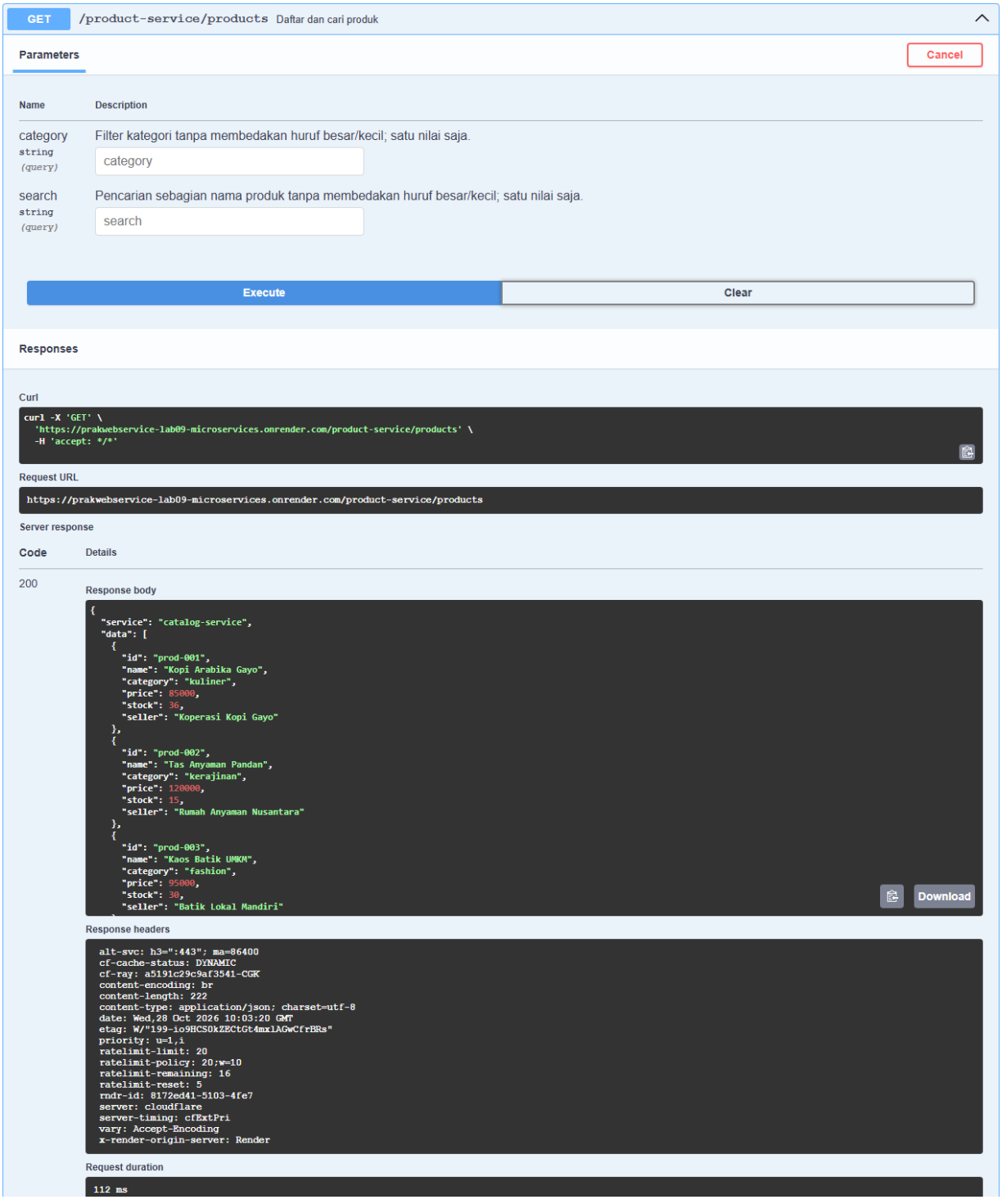
Transaksi melalui frontend menambahkan pesanan dua kopi senilai Rp170.000. Riwayat menampilkan pesanan dari verifier HTTP dan pesanan frontend berikutnya; stok berubah dari 38 menjadi 36 pada transaksi UI.



Gambar 12. Riwayat pesanan publik sesudah checkout frontend berhasil. Nilai API key disembunyikan dan hanya digunakan dalam sesi halaman.

6.10 Swagger Try it out pada deployment publik

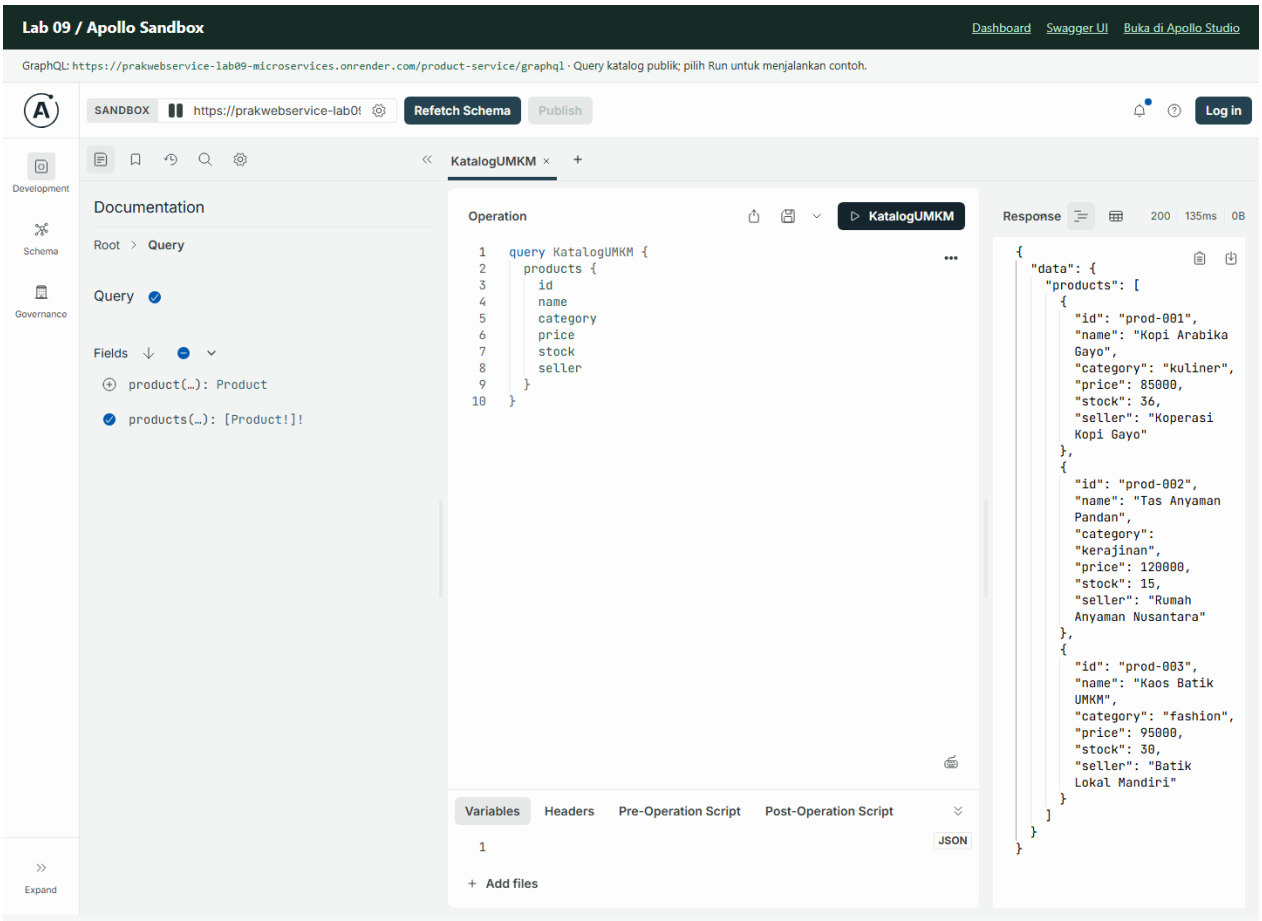
Swagger yang di-deploy menjalankan `GET` ke katalog melalui URL HTTPS gateway. Bagian Request URL dan respons server berikut berasal dari eksekusi publik, bukan dari example value statis OpenAPI.



Gambar 13. Swagger UI publik: request HTTPS, respons HTTP 200, JSON katalog, dan header server pada service Render Lab 09.

6.11 Apollo Sandbox pada deployment publik

Operasi `KatalogUMKM` dijalankan dari tombol Run pada Sandbox yang di-deploy. Endpoint yang terlihat di bagian atas adalah GraphQL HTTPS gateway Lab 09. Respons HTTP 200 memuat tiga produk; stok kopi 36 sesuai transaksi frontend sebelumnya.



Gambar 14. Apollo Sandbox publik menampilkan query dan data respons. Rekaman HTTP aktual tersedia pada apollo-query.json.

Respons tersebut dicatat pada `docs/evidence/deployment/apollo-query.json` pada 28 Oktober 2026 pukul 17:03:29 UTC+7. Dengan demikian, dokumentasi GraphQL publik tidak hanya dapat dibuka, tetapi juga berhasil menjalankan query katalog.

7. Batasan konsistensi dan refleksi

7.1 Konsistensi data

Validasi dan pengurangan stok dilakukan sinkron pada satu proses katalog, tanpa jeda `await` di antara pemeriksaan dan mutasi. Produk berulang digabungkan sehingga stok tidak terpotong berdasarkan pemeriksaan yang terpisah. Batch yang ditolak tidak mengubah sebagian stok.

Jaminan tersebut terbatas pada operasi dalam katalog. Pengurangan stok dan penyimpanan pesanan pada service lain belum menjadi transaksi terdistribusi. Jika reservasi berhasil tetapi Order Service gagal menyimpan pesanan, stok dapat berkurang tanpa pesanan yang terlihat oleh klien. Timeout juga tidak membuktikan katalog belum memproses request.

Implementasi masih memakai memori; restart menghapus data proses terkait. Idempotency key, masa berlaku reservasi, operasi kompensasi, database persisten, dan koordinasi pembayaran merupakan pengembangan berikutnya.

7.2 Refleksi

Ketika Catalog Service mati, Order Service masih dapat menampilkan pesanan karena riwayat pesanan dimiliki dan dibaca dari memorinya sendiri. Hal ini memungkinkan pengguna memeriksa transaksi yang sudah tersimpan sambil menunggu katalog pulih. Jika stok telah dikurangi oleh reservasi tetapi pembuatan pesanan gagal sesudahnya, stok dan pesanan dapat menjadi tidak konsisten sehingga perlu mekanisme kompensasi atau reservasi dengan masa berlaku. Dibanding monolit, microservices menambah panggilan jaringan, timeout, kemungkinan kegagalan sebagian, serta pekerjaan konfigurasi dan pengujian. Pemisahan tanggung jawab membantu membatasi dampak gangguan, tetapi manfaat itu harus dibayar dengan pengelolaan konsistensi dan operasi yang lebih rumit.

7.3 Referensi dan berkas pengumpulan

- Materi praktikum: *Lab 09: Memecah Aplikasi Menjadi Microservices*, Web Service, Philipus Suryo Subandoro.
- Repository implementasi: prakwebservice-lab09-microservices.
- Asal gateway: repository Lab 08.
- Bukti mesin dan asal screenshot: `docs/evidence/lab09/`.
- Collection Postman JSON: `docs/postman/`.
- Sumber LaTeX mandiri dan gambar: `docs/latex-lab09/`.